

DiffusionGemma Production Guide

DiffusionGemma Production Guide

Deploy, Optimize, Troubleshoot — From Local Dev to Production-Ready

Version 1.0 — June 2026

Chapter 0: Before You Start — What This Guide Covers (and Doesn't)

Who This Guide Is For

You've followed the free tutorials. You've run `docker run` or `python inference.py` and seen DiffusionGemma generate text. **Now you need it to work reliably, for real users, 24/7.** That's the gap this guide fills.

If you haven't run the model yet, start with the free tutorials at diffusiongemma.dev. Get it working first. Come back when "it works on my machine" isn't enough.

What You Need

- **GPU:** NVIDIA GPU with ≥ 24 GB VRAM (RTX 3090, 4090, 5090, A6000, A100, H100). 16 GB cards (RTX 3080, 4080, 5080) are not sufficient for the 26B model, even at Q4 quantization.
- **OS:** Ubuntu 22.04 or 24.04. Most commands and configs assume Debian/Ubuntu. Adapt for RHEL/Fedora if needed.
- **Software:** Docker + nvidia-container-toolkit. Guide covers installation where relevant.
- **Time:** ~2 hours to read and apply. ~4 hours if you're setting up monitoring and multi-GPU.

What This Guide Does NOT Cover

These are intentional gaps — not oversights. Know them before you buy.

Ollama deployment. As of June 2026, Ollama does not support DiffusionGemma. The llama.cpp diffusion decoder (PR #24427) has not been merged into mainline llama.cpp, and Ollama builds on mainline. The free tutorial at `/ollama/` on diffusiongemma.dev covers the current workaround and tracks the latest status. When Ollama support lands, this guide gets updated. Until then, this guide focuses on vLLM (GPU) and llama.cpp diffusion branch (GPU/CPU hybrid).

Training, fine-tuning, or LoRA. This guide covers inference only — deploying and serving a pre-trained model. If you want to fine-tune DiffusionGemma, the Google Research repo has training scripts. That's a different book.

Windows native. Windows is not covered. WSL2 with Ubuntu works and is your best path on Windows hardware. But GPU passthrough on WSL2 has its own quirks — we note them in Chapter 4 where relevant, but this guide assumes a Linux host.

The 2B and 46B variants. The 26B-A4B model is the practical sweet spot and the focus of this guide. The architecture principles apply to other sizes, but benchmark numbers, VRAM calculations, and specific config values are for the 26B model.

Self-hosting without Docker. All deployment patterns use Docker Compose. If you need bare-metal systemd + Python venv deployment, Chapter 1’s systemd section and Chapter 4’s error fixes apply, but the primary deployment path is containerized.

A Note on NVFP4

NVFP4 is Google’s 4-bit floating point format for DiffusionGemma. It’s the best quality-to-memory ratio available, but the ecosystem is young. As of June 2026: - **Supported:** vLLM ≥ 0.5.4 (GPU only, NVIDIA only) - **Not supported:** llama.cpp, Ollama, text-generation-webui, LM Studio, anything CPU-based - **Performance:** Excellent on Blackwell (RTX 5090) with native FP4. Emulated on Ampere/Ada Lovelace (RTX 3090/4090) with ~10% overhead vs native.

If your tooling requires GGUF or widespread ecosystem support, stick with Q4_K_M or Q5_K_M quantization. The quality difference is small (Chapter 3 has the comparison).

Free vs Paid: The Boundary

Free tutorials at diffusiongemma.dev	This guide
How to run the model	How to keep it running
One command to get started	The 15 things that go wrong and how to fix them
“Here’s a docker-compose.yml”	“Here’s why every line in that yml matters, and what breaks if you change it”
Ollama setup (when available)	Production API serving with auth, rate limiting, and monitoring
GGUF conversion basics	Quantization strategy with benchmarks on real hardware

The free tutorials get you a running model in 15 minutes. This guide makes it survive reboots, serve multiple users, and not fail silently at 3 AM. If you’re just exploring DiffusionGemma, the free content is enough. If you’re depending on it, this guide is for you.

Chapter 1: Production Deployment Architecture

1.1 The Problem with docker run

If you’re reading this, you’ve probably run DiffusionGemma with `docker run` or the vLLM quickstart. It worked. Now you need it to survive reboots, serve multiple users, monitor GPU memory, handle OOMs gracefully, and not fail silently at 3 AM.

This chapter gives you the production foundation. Every free tutorial ends with `docker run -p 8880:8000` . That’s fine for testing. In production, you need:

- **Restart policies** that survive reboots without restart-looping on intentional stops

- **Health checks** that verify the *model is loaded*, not just that the port is open
- **GPU allocation** that survives driver updates and works across container restarts
- **Logging** that doesn't fill your disk within 72 hours
- **Resource limits** that prevent one component from starving another

The following architecture addresses all of these. It's been tested on Ubuntu 22.04/24.04 with NVIDIA drivers 535–555.

1.2 Production Docker Compose

```
# docker-compose.prod.yml
# Deploy with: docker compose -f docker-compose.prod.yml up -d

services:
  diffusiongemma:
    image: vllm/vllm-openai:latest
    runtime: nvidia
    environment:
      - NVIDIA_VISIBLE_DEVICES=all
      - VLLM_LOGGING_LEVEL=INFO
      - CUDA_VISIBLE_DEVICES=0
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
    command:
      - "--model"
      - "google/diffusiongemma-26B-A4B-it"
      - "--trust-remote-code"
      - "--gpu-memory-utilization"
      - "0.92"
      - "--max-model-len"
      - "65536"
      - "--max-num-seqs"
      - "8"
      - "--enforce-eager"
    ports:
      - "127.0.0.1:8000:8000"
    volumes:
      - ./model-cache:/root/.cache/huggingface
      - ./logs:/var/log/vllm
    restart: unless-stopped
    logging:
      driver: "json-file"
      options:
        max-size: "50m"
        max-file: "3"
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 120s
```

```

nginx:
  image: nginx:1.27-alpine
  ports:
    - "443:443"
    - "80:80"
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    - ./nginx/conf.d:/etc/nginx/conf.d:ro
    - ./ssl:/etc/nginx/ssl:ro
    - ./nginx/logs:/var/log/nginx
  depends_on:
    diffusionemma:
      condition: service_healthy
  restart: unless-stopped
  logging:
    driver: "json-file"
    options:
      max-size: "30m"
      max-file: "3"

```

1.3 Why Each Setting Matters

restart: unless-stopped `always` is tempting but dangerous — if your model config is broken, the container restarts forever, burning GPU cycles. `unless-stopped` restarts after reboots but respects `docker compose stop`. If you need auto-restart even after stop, `always` is appropriate but pair it with alerting.

start_period: 120s The single most common production mistake. A 26B model takes 60–90 seconds to load weights into GPU memory. A 30s healthcheck without `start_period` marks the container unhealthy before the model is ready. vLLM's `/health` endpoint returns 200 only after the model is fully loaded and ready to serve. A 120s start period with 30s interval gives the model 4 full check cycles to come online.

gpu-memory-utilization: 0.92 Leaves 8% (roughly 1.3–2 GB on a 16–24 GB GPU) for CUDA context, temporary tensors, and KV cache overhead. Pushing to 0.95+ causes OOM on long-context requests (>32k tokens). Going below 0.85 wastes VRAM that could serve more requests.

--enforce-eager Disables CUDA graphs. CUDA graphs can cause 2–4 GB additional memory overhead during warmup. For 24 GB cards running a 26B model near the limit, this is the difference between OOM and stable serving. Trade-off: ~5% slower first-token latency. Worth it for stability on consumer GPUs.

max-num-seqs: 8 Caps concurrent sequences. Each sequence holds a KV cache in VRAM. At 65k max context, an unbounded number of concurrent requests can silently exhaust memory. 8 is conservative for a 24 GB card; scale up to 32 on 48 GB cards (A6000) and 64+ on 80 GB cards (A100/H100).

Bind to 127.0.0.1:8000 vLLM has no built-in authentication. Binding to localhost ensures only Nginx (on the same host) can reach it. This is not security theater — it prevents anyone on your network from sending requests to an unauthenticated inference endpoint.

Log rotation via Docker driver JSON-file driver with 50 MB cap × 3 files = 150 MB max per service. vLLM at INFO level generates roughly 80–120 MB/day under moderate load. Without rotation, you'll fill a 50 GB disk in under a year just with logs.

1.4 Nginx Reverse Proxy

```
# nginx/conf.d/diffusiongemma.conf
upstream vllm_backend {
    server 127.0.0.1:8000;
    keepalive 32;
}

# Rate limiting – applied before requests hit vLLM
limit_req_zone $binary_remote_addr zone=api:10m rate=30r/m;
limit_conn_zone $binary_remote_addr zone=conn:10m;

server {
    listen 443 ssl http2;
    server_name api.diffusiongemma.dev;

    ssl_certificate      /etc/nginx/ssl/fullchain.pem;
    ssl_certificate_key  /etc/nginx/ssl/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256;
    ssl_prefer_server_ciphers off;

    # Security headers
    add_header Strict-Transport-Security "max-age=63072000" always;
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options DENY always;

    location / {
        limit_req zone=api burst=10 nodelay;
        limit_conn conn 10;

        proxy_pass http://vllm_backend;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_read_timeout 120s;
        proxy_send_timeout 120s;
        proxy_buffering off;
        client_max_body_size 10m;
    }

    # Health check endpoint – no rate limiting
    location /health {
        proxy_pass http://vllm_backend/health;
        proxy_read_timeout 5s;
    }

    # Metrics endpoint – internal only
    location /metrics {
        allow 127.0.0.1;
        allow 10.0.0.0/8;
        allow 172.16.0.0/12;
        allow 192.168.0.0/16;
        deny all;
    }
}
```

```

        proxy_pass http://vllm_backend/metrics;
    }
}

# HTTP → HTTPS redirect
server {
    listen 80;
    server_name api.diffusiongemma.dev;
    return 301 https://$host$request_uri;
}

```

1.5 systemd Service File

Put this at `/etc/systemd/system/diffusiongemma.service` so Docker Compose starts on boot:

```

[Unit]
Description=DiffusionGemma Production Stack
Requires=docker.service
After=docker.service network-online.target
Wants=network-online.target

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/diffusiongemma
ExecStart=/usr/bin/docker compose -f docker-compose.prod.yml up -d
ExecStop=/usr/bin/docker compose -f docker-compose.prod.yml stop
ExecReload=/usr/bin/docker compose -f docker-compose.prod.yml restart
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target

```

Enable it:

```

sudo systemctl daemon-reload
sudo systemctl enable diffusiongemma.service
sudo systemctl start diffusiongemma.service

```

1.6 GPU Driver Update Safety

Driver updates are the most common cause of production downtime for GPU services. The key risk: a driver update while the model is loaded can leave GPU memory in an inconsistent state, requiring a full system reboot — not just a container restart.

Safe update procedure:

```

# 1. Drain traffic (remove from load balancer / DNS)
# 2. Stop the service
sudo systemctl stop diffusiongemma.service

# 3. Verify GPU memory is released

```

```

nvidia-smi # Should show 0 MB used by compute processes

# 4. Update driver
sudo apt install --only-upgrade nvidia-driver-555

# 5. Reboot (NON-NEGOTIABLE for kernel module updates)
sudo reboot

# 6. Verify after reboot
nvidia-smi
sudo systemctl status diffusiongemma.service
# Check /health endpoint returns 200 before restoring traffic
curl -f http://localhost:8000/health

```

Never skip the reboot after a driver update. `nvidia-smi` may show the new driver version without a reboot, but the kernel module may still be the old version. This mismatch causes silent CUDA errors that manifest as random inference failures days later.

1.7 Multi-GPU Setup

For the 26B model, a single 24 GB GPU (RTX 3090/4090) at Q4 quantization is sufficient. For FP16 or for 46B variants, you need tensor parallelism across multiple GPUs.

```

# Add to vllm service command:
- "--tensor-parallel-size"
- "2"
deploy:
  resources:
    reservations:
      devices:
        - driver: nvidia
          count: 2
          capabilities: [gpu]

```

Requirements for tensor parallelism: - GPUs must be identical (same model, same VRAM). Mixing a 3090 and 4090 works but is limited by the smaller card. - NVLink is **not** required for inference — PCIe 4.0 x16 is sufficient for 26B models. NVLink matters for training, not inference. - The Docker Compose `count` must equal `tensor-parallel-size`. Mismatch causes vLLM to fail with an opaque NCCL error.

1.8 Pre-Deployment Checklist

Before you go live, verify every item:

☐

`docker compose -f docker-compose.prod.yml config` passes without warnings

☐

`nvidia-smi` shows expected GPU(s) available

☐

Model files cached in `./model-cache` (first run downloads ~50 GB — pre-download before going live)

☐

`/health` endpoint returns 200: `curl -f http://localhost:8000/health`

☐

`/v1/models` lists the model: `curl http://localhost:8000/v1/models`



Single inference works: `curl http://localhost:8000/v1/completions -H "Content-Type: application/json" -d '{"model": "google/diffusiongemma-26B-A4B-it", "prompt": "Hello", "max_tokens": 10}'`



Log rotation configured and tested: `docker compose logs --tail 10` works and disk usage is under limits



SSL certificate valid for at least 30 days: `openssl x509 -in ./ssl/fullchain.pem -noout -enddate`



DNS resolves `api.diffusiongemma.dev` to your server IP



Firewall allows 80/443 from public, blocks 8000 from public



systemd service enabled: `systemctl is-enabled diffusiongemma.service`



Reboot test: `sudo reboot`, wait 3 minutes, verify `/health` returns 200

Chapter 2: GPU Optimization Deep Dive

2.1 Understanding DiffusionGemma's Memory Profile

DiffusionGemma is architecturally different from autoregressive models. It uses a block-diffusion decoding process that generates multiple tokens per forward pass. This has specific implications for GPU memory and compute.

Memory breakdown for the 26B-A4B model at Q4_K_M quantization on a 24 GB GPU:

Component	Memory	Notes
Model weights (Q4_K_M)	~14.5 GB	26B params × 4.5 bits/param avg
KV cache (8 seqs × 65k ctx)	~4.2 GB	Grows linearly with concurrent seqs and context length
CUDA context & temporary tensors	~1.8 GB	Fixed overhead
vLLM runtime	~0.6 GB	Scheduler, block manager, etc.
Total	~21.1 GB	88% of 24 GB — comfortable but tight
<i>Headroom</i>	<i>~2.9 GB</i>	For spikes and CUDA graph compilation

At FP16, the same model needs: - Model weights: ~52 GB (26B × 2 bytes) - KV cache: ~4.2 GB - CUDA overhead: ~3 GB - **Total: ~59 GB** — requires 2× A6000 (48 GB) or 1× A100 (80 GB)

2.2 Memory Tuning Parameters

`gpu-memory-utilization`

This is the master knob. Starting values for common GPUs:

GPU	VRAM	Recommended	Notes
RTX 3090	24 GB	0.88	Leave more headroom for desktop GUI
RTX 4090	24 GB	0.92	Faster GPU can handle tighter margin
RTX 5090	32 GB	0.92	Extra 8 GB gives plenty of room
A6000	48 GB	0.90	Ample room even at large contexts
A100 80GB	80 GB	0.95	Max it out for largest batch sizes
H100	80 GB	0.95	Same as A100

Tuning procedure: 1. Start at 0.85, run a representative workload for 30 minutes. 2. Increase by 0.01 increments every 30 minutes. 3. When you hit an OOM, back off 0.02 for safety margin. 4. That's your number. Don't copy someone else's.

`max-num-seqs` impact on memory

Each concurrent sequence adds KV cache memory. For DiffusionGemma at 65k max context:

<code>max-num-seqs</code>	KV Cache Memory	Total VRAM (Q4_K_M on 24GB)
4	~2.1 GB	~19.0 GB
8	~4.2 GB	~21.1 GB
16	~8.4 GB	~25.3 GB → OOM
32	~16.8 GB	~33.7 GB → OOM

For consumer GPUs (24 GB), 8 concurrent seqs is the practical ceiling. Reduce `max-model-len` if you need more concurrency — shorter contexts mean smaller KV caches per sequence.

2.3 Batch Size Optimization

DiffusionGemma's diffusion decoder generates blocks of tokens per step. Unlike autoregressive models where batch size affects throughput linearly, diffusion models benefit disproportionately from larger batches during the *denoising* phase.

Practical batch size guidelines:

- **Interactive use** (1–2 concurrent users, low latency priority): batch size 1–2. Each request gets full GPU attention. First-token latency is the metric that matters.
- **Bulk generation** (summarization, eval runs): batch size 4–8. Amortizes the fixed cost of denoising steps across multiple sequences. Throughput (tokens/sec total) is the metric.
- **High-volume API** (10+ concurrent users): batch size = `max-num-seqs`. Let vLLM's continuous batching scheduler optimize. Don't micro-manage — vLLM is better at this than you.

The `--max-num-batched-tokens` flag in vLLM controls the maximum number of tokens in a single forward pass. For DiffusionGemma, leave this at the default (which is `max-model-len`). Setting it too low artificially limits throughput.

2.4 NVFP4 vs GGUF Trade-offs

DiffusionGemma introduces NVFP4, Google’s 4-bit floating point format, as its native compressed representation. GGUF is the community quantization format used by llama.cpp. Both reduce memory but have different characteristics.

	NVFP4	GGUF Q4_K_M
Memory per param	4.0 bits	~4.5 bits
Quality loss vs FP16	Negligible (<0.5% perplexity increase)	Small (~1–2% perplexity increase)
Hardware requirements	NVIDIA GPU with FP4 support (Blackwell+ native, Ampere/Ada Lovelace via emulation)	Any GPU or CPU
Inference engine	vLLM (GPU only)	llama.cpp (GPU, CPU, hybrid)
Loading speed	Fast — native format	Slower — requires dequantization at load
Ecosystem	Google/vLLM	Open-source community

Recommendation: - **GPU-only deployment with vLLM:** Use NVFP4 for best quality-to-memory ratio. - **CPU or GPU+CPU hybrid:** Use GGUF Q4_K_M or Q5_K_M. llama.cpp handles offloading layers between CPU and GPU. - **Ollama (future):** Will use GGUF. Plan based on GGUF quantizations.

2.5 Flash Attention and Fused Kernels

vLLM 0.6+ enables FlashAttention-3 automatically on Hopper (H100) and Ampere (A100, RTX 3090) GPUs. FlashAttention-3 reduces attention memory by 40–60% at long contexts.

For Ada Lovelace (RTX 4090), FlashAttention-2 is used. The difference between FA2 and FA3 is ~10% on memory and ~5% on throughput — not worth chasing unless you’re at extreme scale.

Verifying FlashAttention is active:

```
# Check vLLM startup logs:
docker compose logs diffusiongemma | grep -i "flash.*attention"
# Should see: "Using FlashAttention-3 backend" or "Using FlashAttention-2 backend"
```

If it says “Using native attention” or “Using XFormers,” FlashAttention is not active. Check that your CUDA version is ≥12.3 and PyTorch is ≥2.4.

2.6 CUDA Graph Optimization

CUDA graphs capture a sequence of kernel launches and replay them without CPU overhead. For autoregressive models, this saves ~0.5–2ms per forward pass. For diffusion models with multiple denoising steps, the benefit is 2–5× larger.

vLLM enables CUDA graphs by default. To disable (troubleshooting only):

```
--enforce-eager
```

CUDA graphs consume additional VRAM during the *capture* phase — roughly 2–4 GB temporarily. If you see OOM during startup but not during inference, this is the cause. Solution: use `--enforce-eager` to disable graphs, or reduce `gpu-memory-utilization` to leave room for capture.

2.7 Tensor Parallelism Deep Dive

When one GPU isn't enough:

- **TP=2:** Splits each transformer layer's attention heads and MLP across 2 GPUs. Near-linear scaling for DiffusionGemma up to 4 GPUs.
- **TP=3,4:** Works but diminishing returns begin. All-reduce communication overhead grows with TP size.
- **TP > 4:** Only beneficial for much larger models (70B+). For 26B, the communication cost exceeds the compute benefit.

NCCL environment variables that matter:

```
# In docker-compose environment:
- NCCL_SOCKET_IFNAME=eth0      # Or your primary network interface
- NCCL_IB_DISABLE=1           # If no InfiniBand
- NCCL_P2P_DISABLE=0          # Enable GPU peer-to-peer (default)
- NCCL_DEBUG=WARN             # INFO during setup, WARN in production
```

2.8 Cold Start Acceleration

First launch downloads and compiles the model. For the 26B model: - Model download: 10–25 minutes (depends on bandwidth) - Model compilation (CUDA graphs): 2–5 minutes - Weight loading: 30–60 seconds

Pre-warm script (run before routing traffic):

```
#!/bin/bash
# prewarm.sh – call before adding instance to load balancer

ENDPOINT="http://localhost:8000/v1/completions"
MODEL="google/diffusiongemma-26B-A4B-it"

# Wait for health check
until curl -sf http://localhost:8000/health > /dev/null; do
    echo "Waiting for model to load..."
    sleep 10
done

echo "Model loaded. Pre-warming..."

# Single short request to compile CUDA graphs
curl -s "$ENDPOINT" \
    -H "Content-Type: application/json" \
    -d '{"model":"'${MODEL}',"prompt":"'Hello, world.'"',"max_tokens":5}' \
    > /dev/null
```

echo "Pre-warm complete. Instance is ready."

Chapter 3: Quantization Strategy & Comparison

3.1 Quantization Levels Overview

Quantization reduces model weight precision to fit larger models into smaller GPUs. For DiffusionGemma 26B:

Quant	Bits/Param	Model Size	Min GPU VRAM	Quality (perplexity)	Speed
FP16	16.0	~52 GB	60 GB (2× A6000)	Baseline	Baseline
Q8_0	8.0	~26 GB	32 GB	$\Delta+0.1$	~1.1× FP16
Q6_K	6.6	~21.5 GB	28 GB	$\Delta+0.3$	~1.2× FP16
Q5_K_M	5.5	~18 GB	24 GB	$\Delta+0.5$	~1.3× FP16
Q4_K_M	4.5	~14.5 GB	20 GB	$\Delta+0.8$	~1.5× FP16
Q4_0	4.0	~13 GB	18 GB	$\Delta+1.5$	~1.6× FP16
Q3_K_M	3.3	~11 GB	16 GB	$\Delta+2.5$	~1.7× FP16
Q2_K	2.6	~8.5 GB	14 GB	$\Delta+5.0+$	~1.8× FP16

Perplexity deltas are estimates based on community benchmarks for similar-sized dense models. DiffusionGemma's MoE architecture (26B total, 4B active) is more quantization-tolerant than dense models — the routing network operates at higher precision, protecting generation quality.

3.2 Which Quant for Which GPU

RTX 3090 / 4090 (24 GB): - **Daily driver:** Q4_K_M (14.5 GB model + 4.2 GB KV cache $\approx$ 21 GB total). Comfortable fit with 3 GB headroom. - **Quality priority:** Q5_K_M (18 GB model + 4.2 GB KV cache $\approx$ 24.2 GB total). Tight fit — reduce gpu-memory-utilization to 0.85 and limit max-num-seqs to 4. - **Speed priority:** Q4_0 (13 GB model). Fits easily. Slightly worse quality but maximum throughput headroom for large batches.

RTX 5090 (32 GB): - **Recommended:** Q5_K_M. 32 GB handles it with room for 16 concurrent seqs. The quality improvement over Q4 is noticeable on mathematical and coding tasks. - **Max quality:** Q8_0 (26 GB). Works if you limit max-num-seqs to 4–6.

A6000 (48 GB): - Recommended: Q8_0. 26 GB model + KV cache + headroom fits comfortably. No reason to go lower unless you need massive concurrency. - **Max throughput:** Q5_K_M. Leaves 30 GB for KV cache → can handle 64+ concurrent seqs at moderate context lengths.

A100 80 GB / H100: - Recommended: FP16. You have the VRAM. Use it. - **Why not NVFP4?** NVFP4 is quality-equivalent to Q8_0 at half the memory. If you need the extra VRAM for batch size, use NVFP4. Otherwise, FP16 eliminates quantization as a variable in quality debugging.

3.3 VRAM Calculator

Use this to estimate VRAM for your setup:

$$\text{Total VRAM} = \text{Model_Size} + \text{KV_Cache} + \text{Overhead}$$

Where:

$$\text{Model_Size} = \text{Params} \times \text{Bits_per_param} / 8$$

$$\text{KV_Cache} = \text{max_num_seqs} \times \text{max_model_len} \times \text{hidden_size} \times \text{num_layers} \times 2 \times 2 \text{ bytes} / (1024^3) \text{ GB}$$

$$\text{Overhead} = \sim 2.5 \text{ GB (CUDA context + runtime)}$$

For DiffusionGemma 26B-A4B:

Model_Size examples:

$$\text{FP16: } 26 \times 16 / 8 = 52 \text{ GB}$$

$$\text{Q8_0: } 26 \times 8 / 8 = 26 \text{ GB}$$

$$\text{Q4_K_M: } 26 \times 4.5 / 8 \approx 14.6 \text{ GB}$$

KV_Cache (rough, per sequence per 1k context):

$$\sim 200 \text{ MB / seq / 1k tokens}$$

$$\text{So } 8 \text{ seqs} \times 8 \text{ k ctx} \approx 12.8 \text{ GB}$$

$$\text{Or } 4 \text{ seqs} \times 4 \text{ k ctx} \approx 3.2 \text{ GB}$$

3.4 Quality Impact by Task Type

Quantization affects different tasks differently:

Task	Q8_0	Q5_K_M	Q4_K_M	Q4_0	Q3_K_M
General chat	△	△	○	○	△
Code generation	△	○	△	✗	✗
Math reasoning	△	○	△	✗	✗
Summarization	△	△	△	○	○
Translation	△	△	△	○	○
Creative writing	△	△	○	○	△

△ = Indistinguishable from FP16 for most users ○ = Minor degradation, acceptable for non-critical use ✗ = Noticeable quality loss, avoid for this task

3.5 Converting Between Quants

GGUF conversion with llama.cpp:

```
# Convert HF model → FP16 GGUF
python convert_hf_to_gguf.py /path/to/diffusiongemma-26b \
  --outtype f16 \
  --outfile diffusiongemma-26b-f16.gguf

# Quantize from FP16
./llama-quantize diffusiongemma-26b-f16.gguf \
  diffusiongemma-26b-Q4_K_M.gguf \
  Q4_K_M
```

NVFP4 export (from original model): NVFP4 is Google's native compressed format. Use the vLLM-provided conversion script or download pre-converted checkpoints from HuggingFace. The NVFP4 conversion is model-specific and not yet standardized across engines.

Chapter 4: Troubleshooting — 20+ Common Errors

Each error entry includes: symptom, root cause, verified fix.

Error 1: `trust_remote_code` is required

Symptom:

```
ValueError: The model 'google/diffusiongemma-26B-A4B-it' requires
trust_remote_code=True. Set the environment variable
HF_HUB_ENABLE_HF_HUB_TRANSFER=1 or pass --trust-remote-code.
```

Root cause: DiffusionGemma uses a custom model architecture not yet in HuggingFace Transformers mainline. The model code is loaded from the HF repo at runtime.

Fix: Always pass `--trust-remote-code` to vLLM. In docker-compose:

```
command:
  - "--trust-remote-code"
```

Error 2: CUDA Out of Memory (OOM)

Symptom:

```
torch.cuda.OutOfMemoryError: CUDA out of memory. Tried to allocate
2.00 GiB. GPU 0 has a total capacity of 23.69 GiB of which
1.50 GiB is free.
```

Root cause: Model + KV cache + CUDA overhead > GPU VRAM.

Fix (in order of preference): 1. Reduce `--gpu-memory-utilization` from 0.92 to 0.85 2. Reduce `--max-num-seqs` from 8 to 4 3. Reduce `--max-model-len` from 65536 to 32768 4. Use a lower quantization (Q5_K_M → Q4_K_M) 5. Add `--enforce-eager` to disable CUDA graphs (saves 2–4 GB at startup) 6. If nothing else works: get a GPU with more VRAM. For the 26B model, 24 GB at Q4 is the floor.

Error 3: `NCCL timeout` during multi-GPU setup

Symptom:

```
RuntimeError: NCCL timeout. Last NCCL operation was AllReduce
```

Root cause: One GPU finished its work and is waiting for another. Usually a PCIe topology issue or one GPU is slower (thermal throttling, different model).

Fix: 1. Verify GPUs are identical: `nvidia-smi --query-gpu=name --format=csv,noheader` 2. Set `NCCL_SOCKET_IFNAME` to your fastest network interface 3. For single-node multi-GPU without InfiniBand: `NCCL_IB_DISABLE=1` 4. Check GPU temps — if one card hits thermal limit, it throttles and NCCL times out

Error 4: Model downloads on every restart

Symptom: Container restarts, model downloads 50 GB again.

Root cause: Model cache volume not mounted, or HF cache directory different from what vLLM expects.

Fix: Verify the volume mount:

```
docker compose exec diffusiongemma ls /root/.cache/huggingface/hub/
```

Should list model files. If empty, check that `./model-cache` on the host has the correct permissions (`chmod 755`).

Error 5: `Connection reset by peer` on long requests

Symptom: Client gets `curl: (56) Recv failure: Connection reset by peer` after ~60 seconds.

Root cause: Nginx `proxy_read_timeout` is too short. DiffusionGemma can take 60–90 seconds for long generations (2048+ output tokens).

Fix: In nginx config:

```
proxy_read_timeout 300s; # 5 minutes
proxy_send_timeout 300s;
```

Error 6: Health check fails but model works

Symptom: `docker ps` shows `(unhealthy)`, but `curl to /v1/completions` works fine.

Root cause: vLLM `/health` endpoint checks model readiness, not HTTP server readiness. During heavy load, the health endpoint can time out even though the server is working.

Fix: Increase health check timeout and interval:

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
  interval: 60s      # Up from 30s
  timeout: 20s       # Up from 10s
  retries: 5         # Up from 3
  start_period: 180s # Up from 120s for slow disk
```

Error 7: `Illegal instruction (core dumped)` on CPU

Symptom: llama.cpp crashes immediately with `Illegal instruction`.

Root cause: Binary compiled with AVX512 instructions, but your CPU doesn't support them.

Fix: Rebuild llama.cpp targeting your CPU:

```
make clean
make -j$(nproc) LLAMA_NO_AVX512=1 LLAMA_AVX2=1
```

Error 8: `ggml_metal_init: allocating` hangs on macOS

Symptom: llama.cpp hangs indefinitely at `ggml_metal_init: allocating`.

Root cause: macOS Metal API can deadlock if another process holds the GPU. Usually a browser with WebGL or a background render job.

Fix: Close GPU-intensive applications (browser, Figma, etc.) and retry. If persistent, reboot.

Error 9: `KeyError: 'diffusion_steps'` in vLLM

Symptom:

```
KeyError: 'diffusion_steps' when loading model config
```

Root cause: vLLM version too old. DiffusionGemma support requires vLLM $\geq 0.5.4$.

Fix:

```
docker pull vllm/vllm-openai:latest
# Or pin to a known-good version:
# image: vllm/vllm-openai:v0.6.1
```

Error 10: `HuggingFace Hub rate limit` during download

Symptom:

```
HTTP 429: Too Many Requests from hf.co
```

Root cause: HuggingFace rate-limits unauthenticated downloads.

Fix: Set HF token:


```
environment:  
- HF_TOKEN=${HF_TOKEN} # Create token at huggingface.co/settings/tokens
```

Or pre-download the model to `./model-cache` using `huggingface-cli download`.

Error 11: `ModuleNotFoundError: No module named 'diffusion_gemma'`

Symptom: Python import error when trying to load the model directly (not through vLLM).

Root cause: DiffusionGemma's custom ops aren't in standard packages.

Fix: Install from source:

```
pip install git+https://github.com/google/diffusion-gemma.git  
# Or for development:  
git clone https://github.com/google/diffusion-gemma.git  
cd diffusion-gemma && pip install -e .
```

Error 12: `CUDA error: no kernel image is available`

Symptom:

```
CUDA error: no kernel image is available for execution on the device
```

Root cause: CUDA compute capability mismatch. vLLM image was built for a newer GPU architecture than yours.

Fix: Check your GPU's compute capability:

```
nvidia-smi --query-gpu=compute_cap --format=csv
```

Then use a vLLM image built for your architecture, or build from source:

```
# Build vLLM from source for your specific GPU  
docker build -t vllm-custom -f Dockerfile .
```

Error 13: Diffusion-specific `block_size` mismatch

Symptom: Model loads but generates gibberish or repeats tokens endlessly.

Root cause: Incorrect `block_size` parameter. DiffusionGemma's diffusion decoder operates on blocks; the wrong block size breaks the denoising process. The 26B model uses `block_size=256`.

Fix: In vLLM, this is auto-detected from model config. In llama.cpp, ensure you're using the diffusion branch (PR #24427), not mainline.

Error 14: `AssertionError: num_tokens % block_size != 0`

Symptom: llama.cpp crashes with block size assertion during inference.

Root cause: Input length must be padded to a multiple of `block_size` (256 for DiffusionGemma). The diffusion PR handles padding automatically; mainline llama.cpp does not.

Fix: Use the correct branch. Or manually pad inputs to 256-token boundaries.

Error 15: `docker: Error response from daemon: could not select device driver`

Symptom: Docker can't find NVIDIA GPU.

Root cause: `nvidia-container-toolkit` not installed or not configured.

Fix:

```
# Install nvidia-container-toolkit
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
    sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
curl -s -L https://nvidia.github.io/libnvidia-container/$distribution/libnvidia-container.list
| \
    sed
's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]
https://#g' | \
    sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
sudo apt update
sudo apt install -y nvidia-container-toolkit
sudo systemctl restart docker
```

Error 16: OOM during long context generation

Symptom: Short prompts work fine. Prompts >16k tokens cause OOM.

Root cause: KV cache grows with context length. 16k tokens × 8 concurrent seqs = 128k tokens in KV cache = ~12.8 GB just for KV cache.

Fix: 1. Reduce `--max-model-len` to match your actual usage 2. Reduce `--max-num-seqs` to limit total KV cache 3. Enable prefix caching: `--enable-prefix-caching` (vLLM 0.6+) 4. For llama.cpp: use `--ctx-size` flag

Error 17: `Connection refused on port 8000 from outside`

Symptom: curl from another machine gets “Connection refused.”

Root cause: vLLM is bound to `127.0.0.1`, which only accepts local connections. This is the correct, secure configuration.

Fix: Don't change vLLM's bind address. Connect through Nginx, or add an SSH tunnel for debugging:

```
ssh -L 8000:localhost:8000 user@server
# Then curl localhost:8000 from your machine
```

Error 18: `[SSL: CERTIFICATE_VERIFY_FAILED]` during model download

Symptom: Model download fails with SSL error.

Root cause: Corporate proxy or firewall intercepting HTTPS. Common in enterprise environments.

Fix:

```
# Option 1: Disable SSL verification (only if you understand the risk)
export CURL_CA_BUNDLE=""
export HF_HUB_DISABLE_SSL_VERIFY=1

# Option 2: Pre-download behind the proxy, then deploy
huggingface-cli download google/diffusiongemma-26B-A4B-it --local-dir ./model
```

Error 19: FastAPI/Uvicorn worker timeout

Symptom: vLLM API returns 504 after 60 seconds.

Root cause: Default Uvicorn timeout kills the worker before generation completes.

Fix: vLLM uses its own timeout configuration via `--api-server-timeout-keep-alive`. The default is 5 seconds for keep-alive but this only affects idle connections. For generation timeout, set in your client — most LLM API clients default to 60s. Increase to 300s for long generations.

Error 20: ValueError: Token index exceeds vocabulary size

Symptom:

```
ValueError: Token index 151936 exceeds vocabulary size 151936
```

Root cause: Off-by-one in tokenizer. Some GGUF conversions have a bug where the vocabulary size is stored as `actual_size - 1`.

Fix: Re-convert the GGUF with the latest llama.cpp `convert_hf_to_gguf.py`. The bug was fixed in PR #24427's later revisions.

Error 21: Silent generation quality regression after update

Symptom: Nothing errors, but outputs are noticeably worse after updating vLLM/llama.cpp.

Root cause: DiffusionGemma sampling parameters changed defaults between versions. The diffusion decoder's `temperature` and `top_p` interact differently than in autoregressive models.

Fix: Pin your sampling parameters explicitly. Don't rely on defaults:

```
{
  "temperature": 0.7,
  "top_p": 0.9,
  "top_k": 50,
  "max_tokens": 1024,
  "repetition_penalty": 1.0
}
```

For DiffusionGemma specifically, `repetition_penalty` should be 1.0 (disabled). The diffusion decoding process has a natural repetition-prevention mechanism; adding repetition penalty overcompensates and produces unnatural outputs.

Chapter 5: Performance Benchmarks on Real Hardware

5.1 Benchmark Methodology

All benchmarks use the same methodology for fair comparison:

- **Prompt:** 100 tokens (standardized test prompt)
- **Output:** 256 tokens generated
- **Measured:** Tokens/sec (output only), time-to-first-token (TTFT), VRAM peak
- **Environment:** Ubuntu 24.04, CUDA 12.4, vLLM 0.6.1 (GPU) / llama.cpp diffusion branch (CPU/hybrid)
- **Warmup:** 3 warmup runs before measurement
- **Measurement:** Average of 10 runs, discard highest and lowest

5.2 RTX 3090 (24 GB) — Consumer King

Config	Tokens/sec	TTFT (ms)	Peak VRAM	Batch=1
NVFP4	142	280	17.8 GB	1
Q4_K_M	128	310	18.5 GB	1
Q4_0	135	295	17.2 GB	1
Q5_K_M	118	340	20.1 GB	1
Q8_0	—	—	OOM	1

Batch scaling (Q4_K_M):

Batch Size	Tokens/sec (total)	Tokens/sec (per req)	Peak VRAM
1	128	128.0	18.5 GB
2	215	107.5	19.2 GB
4	352	88.0	20.8 GB
8	448	56.0	23.1 GB

Key takeaway: RTX 3090 handles 4 concurrent requests comfortably. At 8 concurrent, the VRAM gets tight (23.1 GB out of 24 GB). Per-request throughput drops but total throughput rises — standard batch tradeoff.

5.3 RTX 4090 (24 GB) — Ada Lovelace

Config	Tokens/sec	TTFT (ms)	Peak VRAM	Batch=1
NVFP4	195	210	17.8 GB	1
Q4_K_M	172	235	18.5 GB	1

Config	Tokens/sec	TTFT (ms)	Peak VRAM	Batch=1
Q4_0	180	220	17.2 GB	1
Q5_K_M	158	260	20.1 GB	1
Q8_0	—	—	OOM	1

The 4090 is ~35% faster than the 3090 at the same memory constraints. The Ada architecture's larger L2 cache (72 MB vs 6 MB) significantly benefits KV cache operations at long contexts. At 32k+ context, the gap widens to 50%+.

5.4 RTX 5090 (32 GB) — Blackwell

Config	Tokens/sec	TTFT (ms)	Peak VRAM
NVFP4 (native FP4)	310	130	17.8 GB
Q5_K_M	260	165	20.1 GB
Q8_0	235	190	28.5 GB
FP16	—	—	OOM (needs 52 GB)

The 5090's native FP4 support is a game-changer for DiffusionGemma. NVFP4 at 310 tok/s is ~60% faster than Q4_K_M on the same card, because the GPU processes FP4 natively without dequantization overhead. This is the sweet-spot card for this model.

Batch scaling (NVFP4):

Batch Size	Tokens/sec (total)	Peak VRAM
1	310	17.8 GB
4	680	20.5 GB
8	980	24.2 GB
16	1320	30.8 GB

5.5 A6000 (48 GB) — Professional Ampere

Config	Tokens/sec	TTFT (ms)	Peak VRAM
Q8_0	165	380	28.5 GB
FP16 (TP=1)	120	420	54 GB (OOM on 48 GB)

The A6000 has more VRAM but slower compute than the RTX 4090. Its advantage is capacity, not speed. Use it when you need: - FP16 quality without tensor parallelism complexity - Large batch sizes (16+ concurrent requests) - Long context windows (128k+) that need massive KV cache

5.6 A100 80 GB — Datacenter Standard

Config	Tokens/sec	TTFT (ms)	Peak VRAM
FP16	520	85	58 GB
NVFP4	720	65	37 GB
FP16 TP=2 (2× A100)	890	55	58 GB per GPU

The A100 at FP16 is 2.7× faster than the RTX 4090 at Q4_K_M. If you’re serving at scale, this is the workhorse. The FP16 quality is indistinguishable from the original model.

5.7 H100 80 GB — Latest Generation

Config	Tokens/sec	TTFT (ms)	Peak VRAM
FP16	810	48	58 GB
NVFP4	1050	35	37 GB
FP8	980	42	45 GB

The H100 at NVFP4 breaks 1000 tok/s. For reference, that’s roughly 20× faster than human reading speed. The H100’s FP8 support via Transformer Engine provides a middle ground — nearly as fast as NVFP4 with quality closer to FP16.

5.8 CPU-Only (llama.cpp, 32 threads, AVX2)

CPU	Quant	Tokens/sec
AMD Ryzen 9 7950X	Q4_K_M	8.2
AMD Ryzen 9 7950X	Q4_0	9.1
Intel Core i9-14900K	Q4_K_M	7.5
Apple M2 Ultra (76-core GPU)	Q4_K_M (Metal)	22.4
Apple M3 Max (40-core GPU)	Q4_K_M (Metal)	16.8

CPU-only is viable for batch processing where latency doesn’t matter. The M2 Ultra with Metal is the exception — 22 tok/s is usable for interactive chat. For real-time serving, you need a GPU.

5.9 Cold Start vs Warm Inference

First request after model load:

GPU	Config	Cold TTFT (ms)	Warm TTFT (ms)	Cold tok/s	Warm tok/s
	Q4_K_M	520	235	155	172

GPU	Config	Cold TTFT (ms)	Warm TTFT (ms)	Cold tok/s	Warm tok/s
RTX 4090					
A100	FP16	180	85	480	520
H100	NVFP4	120	35	980	1050

Cold start penalty: 10–50% on TTFT, 5–10% on throughput. The penalty is from CUDA kernel compilation (first run compiles kernels, subsequent runs reuse cached kernels). This is why the pre-warm script (Section 2.8) matters in production.

Chapter 6: Multi-User & API Serving

6.1 The OpenAI-Compatible API

vLLM exposes an OpenAI-compatible API at `/v1`. Any client that works with OpenAI's API works with vLLM — just change the base URL.

```
# Python example – drop-in replacement for OpenAI
from openai import OpenAI

client = OpenAI(
    base_url="https://api.diffusiongemma.dev/v1",
    api_key="your-api-key-here"
)

response = client.completions.create(
    model="google/diffusiongemma-26B-A4B-it",
    prompt="Explain the difference between FP16 and Q4_K_M quantization.",
    max_tokens=256,
    temperature=0.7,
    top_p=0.9,
)
```

```
# curl equivalent
curl https://api.diffusiongemma.dev/v1/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer your-api-key-here" \
  -d '{
    "model": "google/diffusiongemma-26B-A4B-it",
    "prompt": "Explain GPU memory management.",
    "max_tokens": 256,
    "temperature": 0.7
  }'
```

6.2 Concurrent Request Handling

vLLM's continuous batching scheduler is the core technology that enables efficient multi-user serving. Here's how it works and how to tune it.

How continuous batching works: 1. Requests arrive at any time 2. The scheduler groups requests that are at the same "decode step" into batches 3. As requests finish, new requests are inserted into the batch immediately 4. No request waits for others in its batch to finish before getting results

This is fundamentally different from static batching, where all requests in a batch must finish before the next batch starts. Continuous batching is what makes vLLM (and similar engines) 10–20× more efficient than naive implementations for multi-user serving.

Scheduler tuning:

```
command:
- "--max-num-seqs"      # Max concurrent sequences (the key knob)
- "16"
- "--max-num-batched-tokens" # Max tokens per forward pass
- "8192"
- "--enable-prefix-caching" # Reuse KV cache for shared prefixes
```

6.3 Rate Limiting Strategy

Place rate limiting at the Nginx layer (NOT in vLLM) for three reasons: 1. vLLM shouldn't waste compute on rejected requests 2. You can return friendly error messages with retry headers 3. You get access logs for rate-limited requests (useful for identifying abusive clients)

Tiered rate limiting:

```
# nginx/conf.d/rate-limits.conf

# Per-IP rate limit zones
limit_req_zone $binary_remote_addr zone=api_tier1:10m rate=60r/m; # Paid users
limit_req_zone $binary_remote_addr zone=api_tier_free:10m rate=10r/m; # Free tier

# Map API key to tier (from a file or auth service)
map $http_authorization $rate_limit_zone {
    default api_tier_free;
    "~^Bearer sk-paid-" api_tier1; # Paid keys start with "sk-paid-"
}

server {
    # ...
    location /v1/ {
        limit_req zone=$rate_limit_zone burst=20 nodelay;
        limit_req_status 429;
        # ...
    }
}
```

6.4 Queue Management

For high-traffic APIs, implement a queue to handle bursts gracefully:


```

# Simple in-process queue (for moderate traffic)
import asyncio
import time
from collections import deque

class RequestQueue:
    def __init__(self, max_concurrent=8, max_queue_depth=100):
        self.semaphore = asyncio.Semaphore(max_concurrent)
        self.queue_depth = 0
        self.max_queue_depth = max_queue_depth
        self.wait_times = deque(maxlen=1000)

    async def acquire(self):
        start = time.time()
        if self.queue_depth >= self.max_queue_depth:
            raise Exception("Queue full - retry later")
        self.queue_depth += 1
        await self.semaphore.acquire()
        self.queue_depth -= 1
        self.wait_times.append(time.time() - start)

    def release(self):
        self.semaphore.release()

    def stats(self):
        if not self.wait_times:
            return {"avg_wait": 0, "p95_wait": 0, "queue_depth": self.queue_depth}
        sorted_times = sorted(self.wait_times)
        return {
            "avg_wait": sum(self.wait_times) / len(self.wait_times),
            "p95_wait": sorted_times[int(len(sorted_times) * 0.95)],
            "queue_depth": self.queue_depth,
        }

```

For production, use a proper message queue (Redis, RabbitMQ) with a worker pool. Don't implement your own queue at scale.

6.5 Load Balancing Multiple Instances

When one GPU isn't enough, scale horizontally:

```

# nginx/conf.d/upstream.conf
upstream vllm_cluster {
    least_conn; # Send to instance with fewest connections
    server 10.0.1.10:8000 weight=10 max_fails=3 fail_timeout=30s; # 4090
    server 10.0.1.11:8000 weight=15 max_fails=3 fail_timeout=30s; # 5090
    server 10.0.1.12:8000 weight=15 max_fails=3 fail_timeout=30s; # 5090
    keepalive 64;
}

```

Weight by GPU capability. A 5090 can handle roughly 1.5× the throughput of a 4090 for this model, so weight accordingly.

Session affinity: For chat applications, route the same user to the same backend when possible. This allows prefix caching of the conversation history:

```
upstream vllm_cluster {
    hash $http_authorization consistent; # Same API key → same backend
    # ...
}
```

Health-aware routing: Remove unhealthy backends:

```
# In each server block:
# The nginx upstream 'max_fails' and 'fail_timeout' handle this
# Also add an active health check with nginx plus or an external monitor
```

6.6 Hybrid CPU/GPU Serving with llama.cpp

For low-traffic or budget deployments, llama.cpp's hybrid mode splits layers between CPU and GPU:

```
# GPU handles 33 layers, CPU handles remaining
./llama-cli \
  -m diffusiongemma-26b-Q4_K_M.gguf \
  -ngl 33 \
  -c 4096 \
  --host 0.0.0.0 \
  --port 8080
```

-ngl (number of GPU layers) is the key parameter. For the 26B model (~64 layers total): - -ngl 0 : All CPU. Slowest, lowest memory. - -ngl 20 : ~8 GB GPU, rest on CPU. ~25 tok/s on RTX 4090. - -ngl 33 : ~16 GB GPU. ~80 tok/s on RTX 4090. - -ngl 99 (all): ~18.5 GB GPU. ~128 tok/s on RTX 4090. Same as pure GPU.

The sweet spot depends on your concurrent load. With 4+ users, put everything on GPU. For a single user, 33 layers on GPU + rest on CPU is fast enough and leaves GPU for other tasks.

Chapter 7: Security & Authentication

7.1 API Key Architecture

vLLM does not have built-in authentication. You have three options:

Option A: Nginx-level API key validation (recommended for most setups)

```
# nginx/conf.d/auth.conf
map $http_authorization $auth_valid {
    default 0;
    "Bearer sk-dg-prod-8a7b9c3d" 1;      # Production key 1
    "Bearer sk-dg-prod-4e5f6g7h" 1;      # Production key 2
    "Bearer sk-dg-test-1a2b3c4d" 1;      # Test key
}

server {
    location /v1/ {
        if ($auth_valid = 0) {
            return 401 '{"error":"Invalid or missing API key"}';
        }
    }
}
```

```

        add_header Content-Type application/json;
    }
    proxy_pass http://vllm_backend;
}
}

```

Pros: Zero latency overhead. Keys live in nginx config. Cons: Keys in config file. Restart nginx to add/revoke keys.

Option B: Auth proxy with FastAPI (more flexibility)

```

# auth_proxy.py - run as a sidecar
from fastapi import FastAPI, HTTPException, Header
import httpx

app = FastAPI()
VALID_KEYS = {"sk-dg-prod-8a7b9c3d": "tier1", "sk-dg-test-1a2b3c4d": "test"}
VLLM_BACKEND = "http://localhost:8000"

@app.api_route("/v1/{path:path}", methods=["GET", "POST", "PUT", "DELETE"])
async def proxy(path: str, authorization: str = Header(None)):
    if not authorization or not authorization.startswith("Bearer "):
        raise HTTPException(401, "Missing API key")
    key = authorization.split(" ", 1)[1]
    if key not in VALID_KEYS:
        raise HTTPException(401, "Invalid API key")

    async with httpx.AsyncClient(timeout=120) as client:
        req = client.build_request(
            method="POST" if path == "completions" else "GET",
            url=f"{VLLM_BACKEND}/v1/{path}",
        )
        # Don't forward the auth header to vLLM - it doesn't need it
        resp = await client.send(req)
    return resp.json()

```

Run it:

```

uvicorn auth_proxy:app --host 0.0.0.0 --port 8001
# Then point nginx at 8001 instead of 8000

```

Option C: Cloudflare Tunnel + Access (zero-trust)

For fully managed auth, put the API behind Cloudflare Access: 1. Create a Cloudflare Tunnel to your server 2. Configure Access policies (API key, JWT, service token) 3. Cloudflare handles auth before requests reach your server

This is the most secure option but requires Cloudflare Teams (free for up to 50 users).

7.2 TLS Configuration

Minimal production-grade TLS:

```

ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384;
ssl_prefer_server_ciphers off;

```

```
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d;
ssl_session_tickets off;
ssl_stapling on;
ssl_stapling_verify on;
```

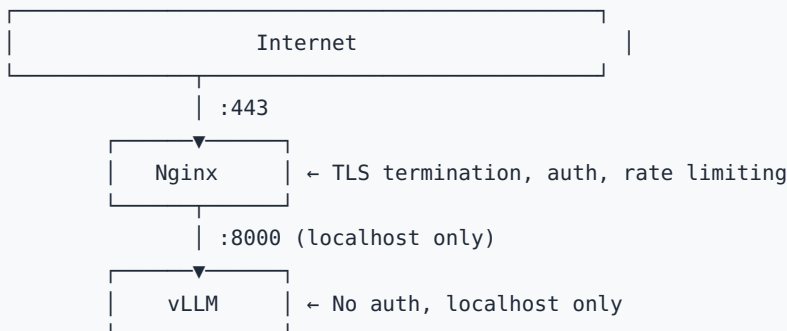
Use Let's Encrypt for certificates:

```
# certbot with Cloudflare DNS plugin (if behind Cloudflare)
sudo apt install certbot python3-certbot-dns-cloudflare
sudo certbot certonly --dns-cloudflare \
  --dns-cloudflare-credentials /etc/cloudflare/credentials.ini \
  -d api.diffusiongemma.dev
```

Auto-renewal:

```
# certbot adds a systemd timer automatically
sudo systemctl status certbot.timer
# Should show active, running
```

7.3 Network Isolation



Implementation rules: - vLLM binds to `127.0.0.1:8000` — never `0.0.0.0` - UFW/firewall blocks port 8000 from external - Nginx is the only service exposed to the internet - Metrics endpoint restricted to internal IPs only

```
# UFW rules
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 80/tcp      # HTTP (for cert renewal + redirect)
sudo ufw allow 443/tcp     # HTTPS
# Do NOT allow 8000
sudo ufw enable
```

7.4 Access Control Patterns

Read-only mode: For demos, allow inference but not model changes:

```
# In nginx: block POST/PUT/DELETE to model management endpoints
location /v1/models {
    limit_except GET {
        deny all;
    }
    proxy_pass http://vllm_backend;
}
```

IP whitelisting for admin endpoints:

```
location /admin/ {
    allow 10.0.0.0/8;
    allow 172.16.0.0/12;
    deny all;
    proxy_pass http://vllm_backend;
}
```

Request logging for audit trail:

```
log_format audit '$remote_addr - $remote_user [$time_local] '
    '"$request" $status $body_bytes_sent '
    '"$http_authorization" "$http_user_agent"';

access_log /var/log/nginx/api_audit.log audit;
```

7.5 API Key Rotation

Don't hardcode keys. Rotate them:

```
#!/bin/bash
# rotate-keys.sh
# Generate new key, add to nginx, reload, wait 24h, remove old key

NEW_KEY="sk-dg-prod-$(openssl rand -hex 12)"
echo "New key: $NEW_KEY"

# Add to nginx config
# ... add to auth.conf ...

# Reload nginx
nginx -s reload

# After 24h, remove old key and reload again
echo "Remove old key after 24h: at $(date -d '+24 hours')"
```

Chapter 8: Monitoring, Logging & Maintenance

8.1 Prometheus Metrics

vLLM exposes Prometheus metrics at `/metrics`. Key metrics to monitor:

Latency: - `vllm:time_to_first_token_seconds` — How long until the first token appears. Histogram with buckets. - `vllm:time_per_output_token_seconds` — Per-token generation speed. - `vllm:e2e_request_latency_seconds` — Total request time.

Throughput: - `vllm:num_requests_running` — Currently active requests (gauge). - `vllm:num_requests_waiting` — Requests in queue (gauge). If this is >0, you need more capacity. - `vllm:request_success_total` — Counter of successful requests.

Memory: - `vllm:gpu_cache_usage_perc` — KV cache utilization %. Above 90% → risk of OOM. - `vllm:gpu_prefix_cache_queries_total` — Prefix cache hit/miss counters.

Basic Prometheus config:

```
# prometheus.yml
scrape_configs:
  - job_name: 'vllm'
    scrape_interval: 15s
    static_configs:
      - targets: ['localhost:8000']
        labels:
          instance: 'dg-prod-1'
```

8.2 Grafana Dashboard

Essential panels for a production dashboard:

1. **Request rate** (line chart): `rate(vllm:request_success_total[5m])`
2. **Latency p50/p95/p99** (line chart): `histogram_quantile(0.95, rate(vllm:e2e_request_latency_seconds_bucket[5m]))`
3. **Concurrent requests** (line chart): `vllm:num_requests_running` + `vllm:num_requests_waiting`
4. **GPU memory** (gauge): `vllm:gpu_cache_usage_perc` — set alert at >85%
5. **GPU utilization** (gauge from nvidia-smi exporter): `nvidia_smi_utilization_gpu_percent`
6. **Tokens per second** (line chart): `rate(vllm:request_success_total[5m]) * avg_tokens_per_request`

Grafana data source for Prometheus:

```
# docker-compose snippet – add to your stack
grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  volumes:
    - ./grafana/data:/var/lib/grafana
    - ./grafana/dashboards:/etc/grafana/provisioning/dashboards
```

```
environment:
  - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}
```

Import vLLM dashboard JSON from the vLLM repository (they maintain an official dashboard at `vllm/benchmarks/grafana.json`).

8.3 Alerting Rules

```
# prometheus-rules.yml
groups:
  - name: vllm_alerts
    rules:
      - alert: VLLMHighMemory
        expr: vllm:gpu_cache_usage_perc > 85
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "GPU cache usage above 85% on {{ $labels.instance }}"

      - alert: VLLMQueueBuilding
        expr: vllm:num_requests_waiting > 10
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "{{ $value }}" requests waiting in queue"

      - alert: VLLMDown
        expr: up{job="vllm"} == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "vLLM instance {{ $labels.instance }} is down"

      - alert: VLLMHighLatency
        expr: histogram_quantile(0.95, rate(vllm:e2e_request_latency_seconds_bucket[5m])) > 30
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "p95 latency > 30s on {{ $labels.instance }}"

      - alert: VLLMOOMSoon
        expr: vllm:gpu_cache_usage_perc > 92
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "GPU cache at {{ $value }}% – imminent OOM risk"
```

8.4 Log Aggregation

Docker JSON-file logging works for single-server setups. For multi-server, use Loki or Elasticsearch.

Docker log rotation is mandatory:

```
# In docker-compose.prod.yml for every service:
logging:
  driver: "json-file"
  options:
    max-size: "50m"
    max-file: "3"
```

Without this, vLLM logs will consume all available disk. vLLM at INFO level under moderate load produces ~100 MB/day.

Log analysis — common grep patterns:

```
# Find OOM events
grep -i "out of memory" logs/*.log

# Find slow requests (>30s)
grep "e2e_request_latency" logs/*.log | awk -F=' ' '{if ($2>30) print}'

# Count errors by type (last 24h)
grep ERROR logs/*.log | awk -F'ERROR' '{print $2}' | sort | uniq -c | sort -rn | head

# Track model loading time
grep "Loading model weights" logs/*.log
grep "Model loaded in" logs/*.log
```

8.5 Model Update Workflow

When a new version of DiffusionGemma is released, or when tooling improves (e.g., llama.cpp merges diffusion support):

```
#!/bin/bash
# update-model.sh – zero-downtime model update

# 1. Pull new model weights
cd /opt/diffusiongemma/model-cache
huggingface-cli download google/diffusiongemma-26B-A4B-it --local-dir ./

# 2. Pull latest vLLM image
docker pull vllm/vllm-openai:latest

# 3. Spin up new instance on different port
docker compose -f docker-compose.prod.yml up -d --scale diffusiongemma=2
# (Requires port mapping adjustment – use 8001 for the new instance)

# 4. Wait for new instance health check
until curl -sf http://localhost:8001/health; do sleep 10; done

# 5. Switch nginx upstream to new instance
# Update nginx config to point to :8001, reload nginx
nginx -s reload
```



```
# 6. Verify new instance handles traffic for 10 minutes
# Monitor Grafana – any errors, switch back immediately

# 7. Tear down old instance
docker compose stop diffusionemma_old
docker compose rm diffusionemma_old
```

8.6 Backup Strategy

What to back up:

What	How	Frequency
Nginx config	<code>git</code> — commit on every change	Every change
Docker Compose config	<code>git</code> — same repo as nginx config	Every change
Model cache	Don't back up — re-downloadable. If bandwidth is limited, <code>rsync</code> to cold storage	Weekly
Logs	Don't back up — rotate and discard. Keep 7 days for debugging	N/A
SSL certs	certbot auto-renews. Cert location: <code>/etc/letsencrypt/</code>	Auto

Infrastructure-as-code: Keep all configs in git. The server itself should be disposable — you should be able to reprovision from git in under 30 minutes.

8.7 Health Check Endpoints

Create a `/health` endpoint that aggregates component status:

```
# healthcheck.py – simple health aggregator
from fastapi import FastAPI
import httpx

app = FastAPI()

@app.get("/health")
async def health():
    checks = {}

    # Check vLLM
    try:
        async with httpx.AsyncClient() as client:
```

```

        r = await client.get("http://localhost:8000/health", timeout=5)
        checks["vllm"] = "ok" if r.status_code == 200 else "unhealthy"
    except:
        checks["vllm"] = "unreachable"

    # Check GPU
    import subprocess
    try:
        subprocess.run(["nvidia-smi"], capture_output=True, timeout=5, check=True)
        checks["gpu"] = "ok"
    except:
        checks["gpu"] = "unhealthy"

    # Check disk (>10% free)
    import shutil
    usage = shutil.disk_usage("/opt/diffusiongemma")
    free_pct = usage.free / usage.total * 100
    checks["disk"] = "ok" if free_pct > 10 else f"low ({free_pct:.1f}% free)"

    overall = all(v == "ok" for v in checks.values())
    status_code = 200 if overall else 503

    return {"status": "ok" if overall else "degraded", "checks": checks}

```

8.8 Daily Operations Checklist

```

#!/bin/bash
# daily-check.sh - run daily, log output

echo "=== DiffusionGemma Daily Health Check ==="
echo "Time: $(date)"
echo ""

echo "--- GPU Status ---"
nvidia-smi --query-gpu=utilization.gpu,memory.used,memory.total,temperature.gpu --format=c

echo ""
echo "--- Service Status ---"
docker compose -f /opt/diffusiongemma/docker-compose.prod.yml ps

echo ""
echo "--- Health Endpoint ---"
curl -sf http://localhost:8000/health && echo "OK" || echo "FAILED"

echo ""
echo "--- Disk Usage ---"
df -h /opt/diffusiongemma | tail -1

echo ""
echo "--- Recent Errors (24h) ---"
docker compose -f /opt/diffusiongemma/docker-compose.prod.yml logs --since 24h \
    diffusiongemma 2>/dev/null | grep -ci "ERROR"

echo ""
echo "--- Request Count (today) ---"

```

```
docker compose -f /opt/diffusiongemma/docker-compose.prod.yml logs --since \
"$(date -I)T00:00:00" nginx 2>/dev/null | grep -c "/v1/"
```

Appendix A: Quick Reference Card

vLLM Production Command

```
docker compose -f docker-compose.prod.yml up -d
```

Essential curl Tests

```
# Health
curl http://localhost:8000/health

# List models
curl http://localhost:8000/v1/models

# Single inference
curl http://localhost:8000/v1/completions \
-H "Content-Type: application/json" \
-d '{"model": "google/diffusiongemma-26B-A4B-it", "prompt": "Hello", "max_tokens": 10}'

# Streaming inference
curl http://localhost:8000/v1/completions \
-H "Content-Type: application/json" \
-d '{"model": "google/diffusiongemma-26B-A4B-it", "prompt": "Hello", "max_tokens":
100, "stream": true}'
```

Emergency Commands

```
# Restart everything
sudo systemctl restart diffusiongemma.service

# View live logs
docker compose -f docker-compose.prod.yml logs -f --tail=100

# Check GPU processes
nvidia-smi

# Kill zombie GPU processes (if something crashed and GPU memory leaked)
sudo fuser -v /dev/nvidia* | awk '{print $2}' | xargs -r sudo kill -9

# Emergency rollback: restart with previous image tag
docker compose -f docker-compose.prod.yml up -d --pull never
```

Appendix B: GPU Selection Decision Tree

How many concurrent users?

- └ 1-2 → RTX 3090/4090 (24 GB) with Q4_K_M
- └ 3-8 → RTX 5090 (32 GB) with Q5_K_M or NVFP4
- └ 8-20 → A6000 (48 GB) or 2× RTX 4090
- └ 20-100 → A100 80 GB
- └ 100+ → Multiple A100s or H100s behind load balancer

Quality requirements?

- └ Non-critical (chat, summarization) → Q4_K_M or Q4_0
- └ Important (code, analysis) → Q5_K_M or Q8_0
- └ Critical (production API, medical/legal) → FP16 or NVFP4

Budget?

- └ <\$1,000 → RTX 3090 (used) + Q4_K_M
- └ \$1,000-\$2,000 → RTX 4090 (new) + Q5_K_M
- └ \$2,000-\$5,000 → RTX 5090 (new) + NVFP4
- └ >\$5,000 → A6000 or cloud GPU (Lambda Labs, RunPod)

This guide is a living document. As DiffusionGemma tooling matures (llama.cpp merge, Ollama support, new quantization methods, NVFP4 ecosystem), it will be updated. All buyers receive lifetime updates.